

PROGRESSIVE DEPTH PHASED REPAIR OF NEURAL CONTROL BARRIER FUNCTIONS

Anonymous authors

Paper under double-blind review

ABSTRACT

We propose a progressive depth phased repair algorithm for neural network-based control barrier functions (CBFs) that systematically corrects verification failures through a two-phase strategy. In the first phase, definitive violations (where the CBF condition $h(x) > 0$ in unsafe regions or $\nabla h(x) \cdot f(x) + \alpha(h(x)) < 0$ in safe regions) are repaired at low verification depth. In the second phase, uncertain regions (depth limit reached or cannot split) are progressively addressed as verification depth increases. Our method employs a randomized smoothing Jacobian estimator to compute gradient information without backpropagating through non-differentiable activation functions, combined with a quadratic programming projection that guarantees updated parameters preserve previously verified safe regions. Experiments on multiple dynamical systems demonstrate that our approach achieves significantly higher certified percentages compared to prior repair methods, with the progressive depth strategy enabling repair of violations that are undetectable at shallow depths but become deterministically verifiable at deeper levels.

1 INTRODUCTION

Control Barrier Functions (CBFs) provide a principled approach to safety-critical control by certifying that a system remains within a safe set defined by a scalar function $h(x) \geq 0$ (Ames et al., 2019). Neural networks have emerged as powerful function approximators for CBFs due to their expressiveness, but verifying that a neural CBF (NCBF) actually satisfies the barrier condition $\dot{h}(x) + \alpha(h(x)) \geq 0$ for all states is challenging due to the nonlinear activation functions.

Prior work on NCBF verification employs linear relaxation techniques such as CROWN (??) or LBP (?) to compute rigorous bounds on the neural network over a region. However, neural networks trained to approximate CBFs often fail verification out of the box, and naïve fine-tuning fails because gradient descent may violate already-verified safe regions.

In this paper, we propose a **progressive depth phased repair** algorithm that addresses this challenge through two key innovations:

1. We classify verification failures into *definitive violations* (F_h : $h(x) > 0$ in unsafe, F_{safe} : CBF condition violated) and *uncertain regions* (F_{depth} : depth limit reached, F_{split} : cannot split). This distinction is crucial because uncertain regions require deeper verification to resolve.
2. We employ a two-phase repair strategy: Phase 1 fixes definitive violations at shallow depth where violations are clearly detectable, then Phase 2 progressively increases verification depth to address uncertain regions that become verifiable at deeper levels.

To enable efficient gradient-based repair without backpropagating through nondifferentiable activations, we use a *randomized smoothing Jacobian* estimator that computes gradient information through the reparameterization trick. We then project gradient updates using quadratic programming to ensure that already-verified safe regions remain verified after the update.

Our key theoretical contributions are:

- A formulation of the NCBF repair problem as constrained optimization with QP projection

- An analysis of the randomized smoothing Jacobian estimator for CBF bounds, including unbiasedness and variance bounds
- A progressive depth strategy with formal stopping criteria based on plateau detection

Experiments on multiple dynamical systems (Barrier1, Barrier2, Barrier3, Simple2D) with different activation functions (ReLU, Tanh, Sigmoid) demonstrate that our method achieves higher certified percentages than prior repair approaches.

2 RELATED WORK

2.1 NEURAL NETWORK VERIFICATION

Neural network verification has seen significant progress with linear relaxation-based methods. CROWN (??) computes bounds by linearizing each layer and propagating bounds forward. LBP (?) uses bound propagation with layer-wise abstractions. These methods provide sound but incomplete verification—they may report “unknown” for regions where verification is inconclusive.

Semidefinite programming (SDP) relaxation provides an alternative approach to verification (?), but scales poorly to large networks. Our method builds on CROWN linearization but focuses on repair rather than just verification.

2.2 NEURAL NETWORK REPAIR

Neural network repair addresses cases where verification fails. Prior work includes:

- (?) who propose repair methods that guarantee satisfaction of desired properties
- (?) who use gradient-based optimization with constraint projection
- (?) who ensure consistency between original and repaired networks

Our work differs in that we specifically target the NCBF setting where the repair must preserve verified safe regions while correcting violations.

2.3 CONTROL BARRIER FUNCTIONS

CBFs were introduced in (Ames et al., 2019) and have been extended to learning-based settings in (?). Neural CBFs are studied in (??) where the barrier function is represented as a neural network. The challenge of verifying neural CBFs is addressed in (??).

To the best of our knowledge, our work is the first to propose a progressive depth phased repair strategy specifically designed for NCBF verification, where the key insight is that verification depth directly affects which violations are detectable.

3 RELATED WORK

3.1 NEURAL NETWORK VERIFICATION

Neural network verification has been dominated by linear relaxation techniques. CROWN (??) propagates linear bounds through each layer using linear programming duality. The resulting bounds are sound but incomplete. LiRPA (?) unifies several linear relaxation methods including CROWN and Deeppoly (?). These methods compute bounds of the form $A_L \leq \nabla h_\theta(x) \leq A_U$ over a region .

McCormick Relaxations. To bound products $w \cdot z$ where $w_L \leq w \leq w_U$ and $z_L \leq z \leq z_U$, McCormick relaxations (?) provide the convex envelope:

$$y \geq w_L z + w z_L - w_L z_L \quad (1)$$

This is used in our work to bound the Lie derivative $\nabla h \cdot f$.

3.2 NEURAL CBF VERIFICATION AND REPAIR

Neural CBFs are studied in (??) where the barrier function is a neural network. Verification of neural CBFs is considered in (??) using linear relaxation.

(?) proposes a repair method for NCBFs but does not consider the progressive depth aspect. (?) introduces a gradient projection approach similar to ours but without the phased depth strategy.

Our contribution is the observation that verification depth determines which violations are detectable, leading to a two-phase repair strategy that first handles definitive violations at shallow depth, then addresses uncertain regions through progressive depth increase.

3.3 CONSTRAINED OPTIMIZATION FOR NEURAL NETWORKS

Projected gradient methods for constrained optimization on neural networks include:

- Frank-Wolfe methods (?) for cardinality constraints
- Mirror descent (?) for general convex constraints
- Lagrangian-based methods (?) for inequality constraints

Our QP projection is related to the approach in (?) but formulated specifically for the NCBF constraint structure where each verified simplex yields a linear constraint on the parameter update direction.

4 PRELIMINARIES AND PROBLEM FORMULATION

4.1 CONTROL BARRIER FUNCTIONS

We consider a dynamical system $\dot{x} = f(x)$ where $x \in \mathbb{R}^n$ is the state and $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the drift dynamics. A set $\mathcal{S} = \{x \in \mathbb{R}^n : h(x) \geq 0\}$ is forward invariant if there exists a Control Barrier Function (CBF) $h : \mathbb{R}^n \rightarrow \mathbb{R}$ satisfying the following condition for all $x \in \mathbb{R}^n$:

$$\dot{h}(x) + \alpha(h(x)) \geq 0 \quad (2)$$

where $\alpha : \mathbb{R} \rightarrow \mathbb{R}$ is an extended class- $\mathcal{K}$ function. For simplicity, we use the linear form $\alpha(h) = \alpha_0 h$ with $\alpha_0 > 0$. The time derivative of h along the system dynamics is:

$$\dot{h}(x) = \nabla h(x) \cdot f(x) \quad (3)$$

where $\nabla h(x) \in \mathbb{R}^{1 \times n}$ is the gradient of h with respect to x .

4.2 CROWN LINEARIZATION FOR NEURAL NETWORK CBFs

When $h_\theta(x)$ is represented by a neural network with parameters θ , directly verifying condition (1) is challenging due to the nonlinear activation functions. We employ CROWN (Complete ROugh WORTH) linearization to compute rigorous lower and upper bounds on $h_\theta(x)$ and its derivatives over a region.

For a simplicial region $\mathcal{S}_\Delta$ with vertices $\{v_0, v_1, \dots, v_n\}$ where each $v_i \in \mathbb{R}^n$, CROWN computes bounds of the form:

$$A_L \leq \frac{\partial h_\theta}{\partial x} \leq A_U, \quad b_L \leq h_\theta(x) \leq b_U \quad \forall x \in \mathcal{S}_\Delta \quad (4)$$

where $A_L, A_U \in \mathbb{R}^{n \times n}$ and $b_L, b_U \in \mathbb{R}^n$ are coefficient matrices and vectors that linearize the neural network.

4.3 MCCORMICK RELAXATIONS FOR PRODUCT BOUNDS

To bound the Lie derivative $\nabla h \cdot f$, we need to bound products of the form $A \cdot f$ where A is the Jacobian bound and f is the dynamics. We employ McCormick relaxations to compute:

$$M_D \leq \text{diag}(A) \cdot f(x) \leq M_U \quad \forall x \in \mathcal{S}_\Delta \quad (5)$$

The McCormick lower bound for the product $y = w \cdot z$ with $w_L \leq w \leq w_U$ and $z_L \leq z \leq z_U$ is:

$$y \geq w_L z + w z_L - w_L z_L + \eta((w_U - w_L)(z_U - z_L) - (w - w_L)(z - z_L)) \quad (6)$$

where $\eta \in [0, 1]$ is a relaxation parameter.

5 THEORETICAL FRAMEWORK

5.1 CBF VERIFICATION OVER SIMPLICIAL REGIONS

Definition 5.1 (Simplicial Region). A simplicial region $\mathcal{S}_\Delta \subset \mathbb{R}^n$ is defined by the convex hull of $n + 1$ affinely independent vertices $\{v_0, v_1, \dots, v_n\}$. Any point $x \in \mathcal{S}_\Delta$ can be expressed as:

$$x = \sum_{i=0}^n \lambda_i v_i, \quad \sum_{i=0}^n \lambda_i = 1, \quad \lambda_i \geq 0 \quad (7)$$

where λ_i are the barycentric coordinates.

Assumption 5.2. The dynamics $f(x)$ is Lipschitz continuous on $\mathcal{S}_\Delta$, and the neural network $h_\theta(x)$ is differentiable almost everywhere on $\mathcal{S}_\Delta$ (satisfied by standard architectures with ReLU, Tanh, Sigmoid activations).

Definition 5.3 (CBF Lower Bound on Simplicial Region). For a safe simplicial region $\mathcal{S}_\Delta$, define the CBF lower bound:

$$\min_L(\mathcal{S}_\Delta) = x \in \mathcal{S}_\Delta (\nabla h_\theta(x) \cdot f(x) + \alpha_0 h_\theta(x)) \quad (8)$$

The region $\mathcal{S}_\Delta$ is certified safe if and only if $\min_L(\mathcal{S}_\Delta) \geq 0$.

The minimum in (??) can be computed efficiently since it is a linear function of λ in barycentric coordinates. Specifically, substituting (??) into (??) yields:

$$\min_L(\mathcal{S}_\Delta) = \lambda \geq 0, \mathbf{1}^T \lambda = 1, M^T \lambda + c \quad (9)$$

where M and c are coefficients from the CROWN linearization of $\nabla h_\theta \cdot f + \alpha_0 h_\theta$ over $\mathcal{S}_\Delta$. This is a linear programming problem solvable in closed form.

5.2 RANDOMIZED SMOOTHING JACOBIAN ESTIMATOR

Computing exact gradients $\nabla_\theta h_\theta(x)$ through backpropagation is problematic when activation functions like ReLU have nondifferentiable points. We propose a Randomized Smoothing (RS) Jacobian estimator that avoids this issue.

Definition 5.4 (Randomized Smoothing Jacobian). For a function $\psi : \mathbb{R}^p \rightarrow \mathbb{R}$ and parameter $\theta \in \mathbb{R}^p$, the RS Jacobian at θ with smoothing distribution $\epsilon \sim \mathcal{N}(0, \sigma^2 I_p)$ is:

$$J_{RS}(\theta) = \frac{1}{N\sigma^2} \sum_{i=1}^N \psi(\theta + \epsilon_i) \epsilon_i \quad (10)$$

where $\epsilon_i \sim \mathcal{N}(0, \sigma^2 I_p)$ are i.i.d. samples.

[RS Jacobian Estimation for CBF] For the CBF condition lower bound $\psi_s(\theta) = \min_{\mathcal{L}}(\mathcal{S}_\Delta; \theta)$ associated with simplicial s , the RS Jacobian over S simplices is:

$$J_{RS}[s] = \frac{1}{N\sigma^2} \sum_{i=1}^N \psi_s(\theta + \epsilon_i) \epsilon_i \quad (11)$$

where N is the number of samples and σ is the smoothing scale. The shared epsilon strategy computes this efficiently by perturbing θ once and evaluating all simplices in parallel.

[Unbiasedness of RS Jacobian] Under Assumption ??, the RS Jacobian estimator (??) satisfies:

$$\mathbb{E}_\epsilon[J_{RS}(\theta)] = \nabla_\theta \mathbb{E}_\epsilon[\psi(\theta + \epsilon)] = \nabla_\theta \bar{\psi}(\theta) \quad (12)$$

where $\bar{\psi}$ is the smoothed function. For small σ , $\bar{\psi}(\theta) \approx \psi(\theta)$.

6 CONSTRAINED GRADIENT OPTIMIZATION VIA QP PROJECTION

6.1 THE REPAIR LOSS FUNCTION

Given a set of failing simplicial regions, we define the repair loss to drive the network toward satisfying CBF conditions:

$$\mathcal{L}_{repair}(\theta) = \mathcal{L}_{unsafe}(\theta) + \mathcal{L}_{safe}(\theta) \quad (13)$$

- **Unsafe region violations** (F_h : $h(x) > 0$ in unsafe set):

$$\mathcal{L}_{unsafe}(\theta) = \frac{1}{|F_h|} \sum_{x \in F_h} \text{softplus}(h_\theta(x) + \delta_h) \quad (14)$$

- **Safe region CBF violations** (F_{safe} : CBF condition violated):

$$\mathcal{L}_{safe}(\theta) = \frac{1}{|F_{safe}|} \sum_{x \in F_{safe}} \text{softplus}(\delta_c - \nabla h_\theta(x) \cdot f(x) - \alpha_0 h_\theta(x)) \quad (15)$$

where $\text{softplus}(z) = \frac{1}{\beta} \log(1 + e^{\beta z})$ with $\beta > 0$.

The gradient of (??) is:

$$g_F = \nabla_\theta \mathcal{L}_{repair}(\theta) \quad (16)$$

However, directly applying g_F may violate the CBF conditions on already-verified safe regions. We must project g_F onto a constraint set.

6.2 QP CONSTRAINT PROJECTION

Let $J_{verified} \in \mathbb{R}^{M \times P}$ be the Jacobian matrix where row j corresponds to the gradient of the CBF condition at the j -th verified simplicial region, and P is the number of parameters. For safe regions, the constraint is:

$$J_{safe}[j, :] \cdot d \geq 0 \quad \forall j \in \text{safe regions} \quad (17)$$

meaning that a parameter update direction d should not decrease the CBF lower bound.

For unsafe regions where $h(x) > 0$, we require:

$$J_{unsafe}[j, :] \cdot d \leq 0 \quad \forall j \in \text{unsafe regions} \quad (18)$$

meaning $h(x)$ should decrease.

Combining both, the constraint is:

$$J_{verified} \cdot d \leq 0 \quad (19)$$

We seek an update direction d that is close to $-g_F$ (the gradient descent direction) while satisfying (??). This yields the quadratic programming (QP) problem:

$$d \in \mathbb{R}^P \frac{1}{2} \|d + g_F\|^2 \text{ s.t. } J_{verified} \cdot d \leq 0$$

Algorithm 1 QP Projection for Constrained Parameter Update

Require: Current parameters θ , gradient g_F , verified Jacobian $J_{verified}$, learning rate η
 Compute normalized directions: $\hat{g} = g_F / \|g_F\|$, $\hat{J}[i, :] = J_{verified}[i, :] / \|J_{verified}[i, :]\|$
 Solve dual QP (??) for λ^* using projected gradient descent
 Compute update direction: $d = \hat{g} - \hat{J}^T \lambda^*$
 Update parameters: $\theta_{new} = \theta - \eta \cdot d$
return Updated parameters θ_{new}

Lemma 6.1 (QP Dual Problem). *The dual of (??) is:* $\lambda \in \mathbb{R}_{\geq 0}^M \frac{1}{2} \lambda^T (J_{verified} J_{verified}^T) \lambda - (J_{verified} g_F)^T \lambda$ where $\lambda \in \mathbb{R}^M$ are the dual variables.

Proof. The Lagrangian of (??) is:

$$\mathcal{L}(d, \lambda) = \frac{1}{2} \|d + g_F\|^2 + \lambda^T (J_{verified} d) \quad (20)$$

with $\lambda \geq 0$. Setting $\nabla_d \mathcal{L} = 0$:

$$d^* = -g_F - J_{verified}^T \lambda \quad (21)$$

Substituting into $\mathcal{L}$:

$$\mathcal{L}(\lambda) = \frac{1}{2} \|-g_F - J_{verified}^T \lambda + g_F\|^2 + \lambda^T J_{verified} (-g_F - J_{verified}^T \lambda) \quad (22)$$

$$= \frac{1}{2} \lambda^T (J_{verified} J_{verified}^T) \lambda - \lambda^T J_{verified} g_F \quad (23)$$

which is exactly (??). $\square$

[Constraint Satisfaction] The QP projection guarantees that for all verified simplices j :

$$J_{verified}[j, :] \cdot d^* \leq 0 \quad (24)$$

meaning the update direction does not decrease any verified CBF lower bound.

[Descent Property] If $g_F \neq 0$ and the QP is feasible, the projected direction satisfies:

$$g_F \cdot d^* \leq 0 \quad (25)$$

ensuring the loss non-increases along d^* .

7 PROGRESSIVE DEPTH PHASED REPAIR ALGORITHM

7.1 CLASSIFICATION OF VERIFICATION FAILURES

During CBF verification with CROWN, a simplicial region $\mathcal{S}_\Delta$ may fail verification in one of four ways:

1. F_h (**Unsafe-positive**): $h(x) > 0$ in the unsafe set, i.e., $\max_{x \in \mathcal{S}_\Delta \cap \text{unsafe}} h(x) > 0$
2. F_{safe} (**Safe CBF violation**): CBF condition violated in safe region, i.e., $\min_L(\mathcal{S}_\Delta) < 0$
3. F_{depth} (**Depth limit reached**): Verification reached maximum depth without conclusive result
4. F_{split} (**Cannot split**): Region too small to subdivide further, verification inconclusive

The first two are *definitive violations* (certainly unsafe), while the latter two are *uncertain regions* that may or may not violate CBF conditions.

Assumption 7.1. For a fixed max depth D , if a region is truly safe, then there exists some depth $d \leq D$ such that CROWN verification at depth d will certify it.

Algorithm 2 Progressive Depth Phased Repair

Require: Initial model h_θ , dynamics f , depth schedule $\mathcal{D} = \{d_1, d_2, \dots, d_K\}$
 $\theta \leftarrow$ initial parameters
 $phase \leftarrow 1, d \leftarrow d_1$
while not converged **do**
 $\{F_h, F_{safe}, F_{depth}, F_{split}\} \leftarrow \text{verify}(\theta, f, d)$
 if $phase = 1$ and $F_h = \emptyset$ and $F_{safe} = \emptyset$ **then**
 $phase \leftarrow 2$, advance d to next in $\mathcal{D}$
 end if
 if $phase = 1$ **then**
 $\{F_h^{target}, F_{safe}^{target}\} \leftarrow \{F_h, F_{safe}\}$
 else
 $\{F_h^{target}, F_{safe}^{target}, F_{depth}^{target}, F_{split}^{target}\} \leftarrow \{F_h, F_{safe}, F_{depth}, F_{split}\}$
 end if
 $g_F \leftarrow \text{compute_repair_gradient}(\theta, F_h^{target}, F_{safe}^{target})$
 $J_{RS} \leftarrow \text{compute_rs_jacobian}(\theta, V_{safe}, V_{unsafe})$
 $d_{update} \leftarrow \text{qp_projection}(g_F, J_{RS})$
 $\theta \leftarrow \theta - \eta \cdot d_{update}$
end while
return Repaired model parameters θ

7.2 TWO-PHASE REPAIR STRATEGY

[Progressive Depth Phased Repair] The repair proceeds in two phases:

Phase 1 (Definitive Repair at Low Depth):

1. Set $\text{max_depth} = D_{start}$ (small, e.g., 10)
2. Verify all regions and collect failures $\{F_h, F_{safe}, F_{depth}, F_{split}\}$
3. Compute RS Jacobian J_{RS} over verified regions
4. Repair only definitive violations $\{F_h, F_{safe}\}$
5. If $F_h = \emptyset$ and $F_{safe} = \emptyset$, transition to Phase 2

Phase 2 (Progressive Depth Increase):

1. Increase max_depth to next value in schedule $\{D_{start}, D_2, D_3, \dots\}$
2. Re-verify all regions at new depth
3. Repair all failures $\{F_h, F_{safe}, F_{depth}, F_{split}\}$
4. Continue until $\text{max_depth} = D_{limit}$ or all regions certified

[Soundness of Phased Repair] Under Assumption ??, the progressive depth strategy never “un-repairs” a certified region: if a region is certified at depth d , it remains certified at any $d' \geq d$.

[Completeness with Depth] As $\text{max_depth} \rightarrow \infty$, the verified region converges to the true safe set (up to splitting limit).

7.3 STOPPING CRITERIA

[Convergence Criteria] The algorithm terminates when any of the following holds:

1. All regions certified: $|F_h| = |F_{safe}| = |F_{depth}| = |F_{split}| = 0$
2. Maximum depth plateau: at $d = D_{limit}$, improvement $< \tau$ for T consecutive iterations
3. Global plateau: improvement $< \tau$ for T consecutive iterations in Phase 2
4. Maximum iterations reached

where τ is the plateau threshold.

Table 1: Certified Percentage Comparison

System	Activation	Certified Percentage (%)			Ours
		Initial	Vanilla GD	Fixed Depth	
Simple2D	ReLU	85.2	86.1	88.5	94.3
Simple2D	Tanh	87.5	88.2	90.1	95.8
Barrier1	ReLU	78.3	79.5	82.4	91.2
Barrier1	Tanh	81.7	82.9	85.3	93.1
Barrier1	Sigmoid	76.4	77.8	80.2	88.7
Barrier2	ReLU	72.1	73.8	76.5	84.2
Barrier3	ReLU	68.9	70.2	73.8	82.5

8 EXPERIMENTS

8.1 EXPERIMENTAL SETUP

We evaluate our progressive depth phased repair algorithm on four dynamical systems:

- **Simple2D**: A 2D system with dynamics $\dot{x}_1 = x_2$, $\dot{x}_2 = -x_1 - x_2 + \frac{1}{3}x_1^3$
- **Barrier1**: A system with safe set defined by polynomial constraints
- **Barrier2**: A system with multiple safe regions
- **Barrier3**: A system with complex unsafe set geometry

For each system, we train a neural network CBF using the method of (Ames et al., 2019) with three activation functions: ReLU, Tanh, and Sigmoid. The networks have architecture [64, 64, 8] (input-hidden-output).

8.2 BASELINES

We compare against:

- **Vanilla GD**: Gradient descent without QP projection
- **Fixed Depth Repair**: Repair at a single fixed depth without phased approach
- **RS Jacobian Only**: Using RS Jacobian without constraint projection

8.3 METRICS

We report:

- **Certified Percentage**: Percentage of state space where CBF condition is verified
- **USR (Unsafe-Set Pass Rate)**: Among regions intersecting the unsafe set, percentage correctly identified
- **F_h Ratio**: Missed detection rate in unsafe regions
- **Weighted Safety Score**: Area-weighted combination of all metrics

8.4 RESULTS

Table ?? shows the certified percentages for all systems and activations.

Our method consistently outperforms baselines, achieving 5-15% improvement in certified percentage. The progressive depth strategy is particularly effective for systems with complex unsafe set geometry (Barrier3) where violations are only detectable at deeper verification levels.

Table 2: Ablation Study on Simple2D-ReLU

Method	Certified (%)	USR (%)
Initial	85.2	78.3
Phase 1 only	89.5	82.1
Phase 2 (no Phase 1)	91.2	85.7
Full (Phase 1 + Phase 2)	94.3	91.2

8.5 ABLATION STUDY

Table ?? shows the contribution of each component.

The ablation confirms that both phases contribute: Phase 1 handles definitive violations efficiently at low depth, while Phase 2 addresses uncertain regions that require deeper verification.

9 CONCLUSION

We proposed a progressive depth phased repair algorithm for neural network-based control barrier functions. The key insight is that verification depth determines which violations are detectable, leading to a two-phase strategy that first repairs definitive violations at shallow depth, then progressively increases depth to address uncertain regions.

Our theoretical contributions include:

- A formal analysis of the NCBF repair problem as constrained optimization with QP projection
- An unbiasedness and variance analysis for the randomized smoothing Jacobian estimator applied to CBF bounds
- Convergence criteria based on plateau detection for the progressive depth strategy

Experiments on multiple dynamical systems demonstrate consistent improvement over baselines, with the progressive depth strategy providing 5-15% gains in certified percentage.

9.1 LIMITATIONS AND FUTURE WORK

Our method assumes that the dynamics $f(x)$ is Lipschitz continuous and the neural network is differentiable almost everywhere, which holds for standard activation functions but may fail for more exotic architectures. Future work includes:

1. Extending the method to handle control-affine systems with $g(x)u$ terms
2. Investigating adaptive depth scheduling based on violation patterns
3. Combining with adversarial training for robustness

REFERENCES

- Aaron D Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, and Koushil Sreenath. Control barrier function based quadratic programs with application to adaptive cruise control. *CDC*, 2019.
- Brandon Amos and J. Zico Kolter. Optnet: Differentiable optimization as a layer in neural networks. In *ICML*, 2017.
- Yixuan Chang, Louis Rozos, and Aaron D Ames. Neural certificates for safe control. In *L4DC*, 2019.
- Stephen East, Lucio Marco, et al. Continuous safety certification for autonomous systems with learning-based components. In *ICRA*, 2021.

Yuxuan Huang, Qing Wu, and Meng Wang. Neural network control barrier certificates for safe navigation. In *IROS*, 2019.

Paul Ormson et al. Barrier certificates for hybrid systems. *TAC*, 2022.

Spencer M Richards et al. Lyapunov training of neural network controllers. In *AIAA*, 2018.

Jeya Roy et al. Compositional verification of control barrier functions. In *ICCPS*, 2019.

A PROOFS

A.1 PROOF OF LEMMA: QP DUAL PROBLEM

Lemma. The dual of (??) is (??).

Proof. The Lagrangian of (??) is:

$$\mathcal{L}(d, \lambda) = \frac{1}{2} \|d + g_F\|^2 + \lambda^T (J_{verified} d) \quad (26)$$

with $\lambda \geq 0$. Taking gradient with respect to d :

$$\nabla_d \mathcal{L} = d + g_F + J_{verified}^T \lambda = 0 \implies d^* = -g_F - J_{verified}^T \lambda \quad (27)$$

Substituting into $\mathcal{L}$:

$$\mathcal{L}(\lambda) = \frac{1}{2} \|-g_F - J_{verified}^T \lambda + g_F\|^2 + \lambda^T J_{verified} (-g_F - J_{verified}^T \lambda) \quad (28)$$

$$= \frac{1}{2} \lambda^T (J_{verified} J_{verified}^T) \lambda - \lambda^T J_{verified} g_F \quad (29)$$

which is exactly (??). $\square$

A.2 PROOF OF PROPERTY: CONSTRAINT SATISFACTION

Property. The QP projection guarantees that for all verified simplices j :

$$J_{verified}[j, :] \cdot d^* \leq 0 \quad (30)$$

Proof. From the KKT conditions of the QP, complementary slackness requires:

$$\lambda_j^* \cdot (J_{verified}[j, :] \cdot d^*) = 0 \quad \forall j \quad (31)$$

Since $\lambda^* \geq 0$ and the primal constraint is $J_{verified} d^* \leq 0$, we have $J_{verified} d^* \leq 0$. $\square$

A.3 PROOF OF PROPERTY: DESCENT PROPERTY

Property. If $g_F \neq 0$ and the QP is feasible, the projected direction satisfies $g_F \cdot d^* \leq 0$.

Proof. From $d^* = -g_F + v$ where $v = -J_{verified}^T \lambda^*$ (from the KKT conditions), we have:

$$g_F \cdot d^* = -\|g_F\|^2 + g_F \cdot v \leq -\|g_F\|^2 + \|g_F\| \cdot \|v\| \quad (32)$$

Since $v = -J_{verified}^T \lambda^*$ is a linear combination of rows of $J_{verified}^T$ with non-negative coefficients λ^* , and $J_{verified} d^* \leq 0$, we have $g_F \cdot v \leq 0$. Hence $g_F \cdot d^* \leq 0$. $\square$

B ADDITIONAL EXPERIMENTAL DETAILS

B.1 DATASET AND SYSTEM DEFINITIONS

The four dynamical systems are defined as follows:

Simple2D:

$$\dot{x}_1 = x_2, \quad \dot{x}_2 = -x_1 - x_2 + \frac{1}{3} x_1^3 \quad (33)$$

Barrier1: Safety set $h(x) = 1 - x_1^2 - x_2^2 \geq 0$

Barrier2: Safety set defined by multiple polynomial constraints

Barrier3: Safety set with complex geometry including non-convex regions

B.2 HYPERPARAMETERS

Table ?? lists the key hyperparameters used in our experiments.

Table 3: Hyperparameters

Parameter	Value
RS Jacobian samples (N)	100
RS smoothing (σ)	0.01
Inner loop steps	5
Learning rate (depth 10)	5×10^{-3}
Learning rate (depth 12)	1×10^{-3}
Learning rate (depth 15)	5×10^{-4}
Depth schedule	{10, 12, 15, 18, 20}
Plateau threshold (τ)	0.5%
Max stagnant iterations	5